Copy

Testing

When working on a big project, it's a bad idea to write 1000 LOC blindly and then start to compile, hoping your code will pass all the tests magically (If you really make it, please let us know, you definitely should be the next Linus Torvalds!). Instead, test-driven development should be your choice. **We will elaborate on this during the TA tutorial section.**

File structure

Under `src/tests`, each lab (except for lab 0) has its corresponding test directory:

- **"src/tests/threads"**
- **"src/tests/userprog"**
- **"src/tests/vm"**
- **"src/tests/filesys"**

Some labs (like lab3: virtual memory) may contain other labs' test cases, you can see the TEST_SUBDIRS variable defined in its corresponding Make.vars file for the details (e.g. for lab3, the file is src/vm/Make.vars).

Under each lab's test directory, the files can be classified as follows:

- **"foo.c"**
  - **C source code** for `foo` test case
- **"foo.ck"**

    - Perl script for checking your output with the reference, you can find **the desired output** for `foo` test case in it.
- **"Rubric.bar"**
    - `bar` is the name of the testing **set**, and the Rubric file defines which tests this set includes and their assigned scores.
    - For example, `src/tests/threads/Rubric.alam` defines the `alarm` testing set which contains all the tests for "Functionality and robustness of alarm clock" :

```
Functionality and robustness of alarm clock:
4   alarm-single
4   alarm-multiple
4   alarm-simultaneous
4   alarm-priority

1   alarm-zero
1   alarm-negative
```


- **"Grading"**
    - Defines the proportion of testing points of each testing set. For example, `src/tests/threads/Grading` :

```
# Percentage of the testing point total designated for each set of
# tests.

20.0%   tests/threads/Rubric.alarm
40.0%   tests/threads/Rubric.priority
40.0%   tests/threads/Rubric.mlfqs
```

See more in section Grading.

Run all tests

Each project has several tests. To completely test your implementation, invoke `make check` from the project build directory.

- This will build and run each test and **print a "pass" or "fail" message** for each one.
- When a test fails, `make check` also **prints some details of the reason for failure**.
- After running all the tests, `make check` also **prints a summary of the test results**.

> For **project 1**, the tests will probably run faster in **Bochs**.
>
> For **the rest of the projects**, they will run much faster in **QEMU**.
>
> - `make check` will select the faster simulator by default, but you can override its choice by specifying `SIMULATOR=--bochs` or `SIMULATOR=--qemu` on the `make check` command line.

Run an individual test

You can also run an individual test at a time.

- A given test *foo* writes its output to `foo.output`, then a script scores the output as "pass" or "fail" and writes the result to `foo.result`.

To run and grade a single test, `make` the `.result` file explicitly from the build directory.

- e.g. Type `make tests/threads/alarm-multiple.result` in the build directory to test the `alarm-multiple` case.
- If `make` says that **the test result is up-to-date**, but you want to re-run it anyway
  - either delete the `.output` file by hand, e.g. `rm tests/threads/alarm-multiple.output`
  - or run `make clean` to delete all build and test output files.

> See Using GDB to Debug a Single Test to learn how to debug an individual test.

Special options

- By default, each test provides feedback only at completion, not during its run. **** If you prefer, you can **observe the progress of each test** by specifying `VERBOSE=1` on the `make` command line, as in `make check VERBOSE=1`.
- You can also **provide arbitrary options** to the `pintos` run by the tests with `PINTOSOPTS='...'`
  - e.g. `make check PINTOSOPTS='-j 1'` to select a jitter value of 1 (see section Debugging versus Testing).

All of the tests and related files are in pintos/src/tests.

Before we test your submission, we will replace the contents of that directory with a pristine, unmodified copy, to ensure that the correct tests are used.

Thus, you can modify some of the tests if that helps in debugging, but we will run and grade with the original.

All software has bugs, so some of our tests may be flawed.

If you think a test failure comes down to a bug in the test, rather than one in your code, please point it out. We will look at it and fix it if necessary. However, there is a much higher possibility that the test failure is caused by a bug (or many bugs) in your code.

Please don't try to take advantage of our generosity in giving out the whole test suite.

Your code has to work properly in the general case, not just for the test cases we supply. For example, it would be unacceptable to explicitly base the kernel's behavior on the name of the running test case. Such attempts to sidestep the test cases will receive no credit. If you think your solution may be in a gray area here, please ask us about it.

Previous
Debug and Test

Next
Debugging

Last updated 1 year ago